

pourbaix v1.0

User and Expert Manual

Technical documentation

2026-05-10

Abstract

`pourbaix` draws potential-pH diagrams with LuaLaTeX and pgfplots. Lua performs the thermodynamic construction, while pgfplots draws straight boundaries, labels, optional water lines, and optional extra ticks. This v1.0 manual is the English reference documentation for the first public release. It contains a user part, with a detailed description of `\addPourbaix` and all public key-value options, and an expert part, with the data model, the algorithms, and the Lua function reference.

Development note and disclaimer. This package was “vibe-coded” by Christophe Jorssen (christophe.jorssen@gmail.com) with the assistance of ChatGPT 5.5. Users remain responsible for checking thermodynamic data, concentration conventions, and generated diagrams before scientific or pedagogical use. The code is distributed under the LaTeX Project Public License, version 1.3c or later. The work has maintenance status “maintained” and the Current Maintainer is Christophe Jorssen.

Contents

I	User manual	2
1	Installation and compilation	2
2	The main macro: <code>\addPourbaix</code>	2
2.1	Basic examples	2
2.2	Complete key reference for <code>\addPourbaix</code>	3
3	Small examples for key groups	4
3.1	Axis ranges and computational ranges	4
3.2	Color, line width, and styles	5
3.3	Domain label modes	5
3.4	Frontier labels	6
3.5	Extra ticks	6
3.6	Water stability lines	7
3.7	Filtering active species	8
3.8	Superposing the same element at different concentrations	8
4	Boundary concentration conventions	8
4.1	<code>species</code>	8
4.2	<code>element species equal</code>	9
4.3	<code>element element equal</code>	9

5	Public macros for database extension	9
5.1	\PourbaixDeclareElement	9
5.2	\PourbaixAddSpecies	9
5.3	\PourbaixAddAcidBase	9
5.4	\PourbaixAddRedox	10
5.5	\PourbaixAddTemplate	10
5.6	Loading, validating, and inspecting data	12
6	Boundary query macros	12
7	Gallery of diagrams	12
7.1	Built-in diagrams from pourbaix-data.lua	12
7.2	Template-based examples	13
7.2.1	Vanadium	14
7.2.2	Sulfur	25
II	Expert manual	31
8	Architecture	31
9	Thermodynamic data model	31
10	Activities and concentration conventions	31
10.1	Species concentration convention	31
10.2	Analytical concentration with equal species concentrations	31
10.3	Analytical concentration with equal element concentrations	31
11	Acid/base chains	32
12	Dominant acid/base form	32
13	Redox ladder and dismutation	32
14	Derived and virtual redox couples	32
15	Boundary extraction	32
16	Vertical boundaries	32
17	Label placement	33
18	Accessing numerical boundary data	33
19	Template generation and thermodynamic data selection	33
19.1	Parsing and species normalization	33
19.2	Vertical constants	33
19.3	Independent redox constants	34
19.4	Linear thermodynamic graph	34
19.5	Selection of emitted couples	34
19.6	Limitations	34
20	Lua function reference	34
21	Design choices	40

Part I

User manual

1 Installation and compilation

Place the three core files

- `pourbaix.sty`,
- `pourbaix.lua`,
- `pourbaix-data.lua`

in the working directory or in a directory visible to TeX. Compile with **LuaLaTeX**. The package loads `pgfplots`, `pgfkeys`, and `mhchem`.

```
\documentclass[tikz,border=6pt]{standalone}
\usepackage{pourbaix}
\begin{document}
\begin{tikzpicture}
\begin{axis}[pourbaix axis,ymin=-1.2,ymax=2]
\addPourbaix[element=Fe,c=1e-2,color=blue];
\end{axis}
\end{tikzpicture}
\end{document}
```

2 The main macro: `\addPourbaix`

`\addPourbaix` is the central command. It must be used inside a `pgfplots axis`. It computes one diagram and emits the corresponding `\addplot` commands.

```
\begin{axis}[pourbaix axis,ymin=-1.2,ymax=2]
\addPourbaix[element=Fe,c=1e-2,color=blue];
\end{axis}
```

The command is intentionally simple: use `pgfplots` to control the axis geometry and use `\addPourbaix` keys to control the thermodynamic diagram.

2.1 Basic examples

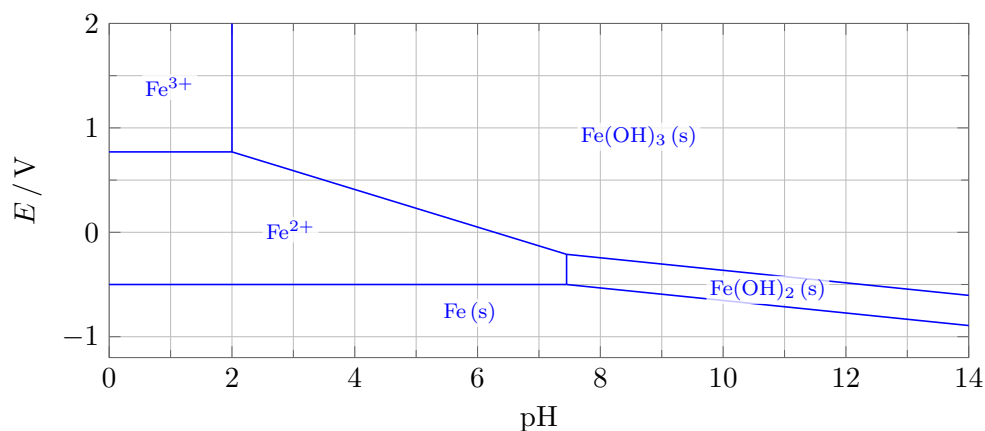


Figure 1: Iron at $c = 10^{-2} \text{ mol L}^{-1}$.

```
\addPourbaix[element=Fe,c=1e-2,color=blue]
```

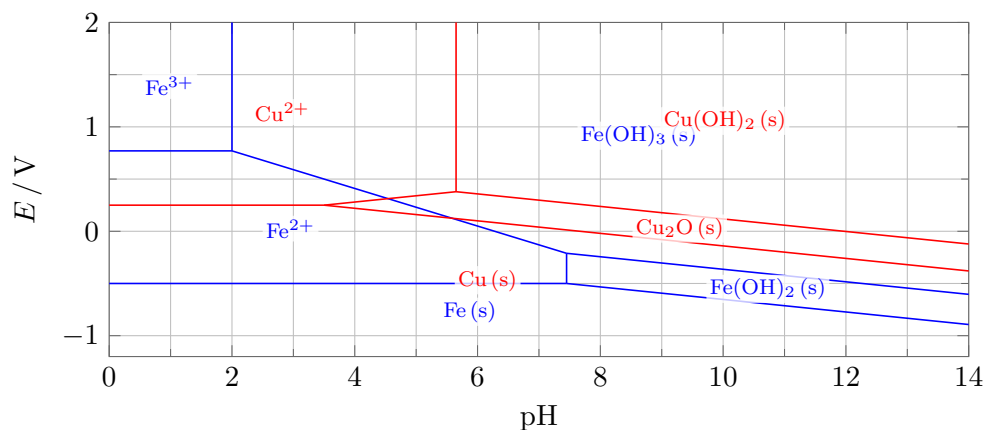


Figure 2: Superposition of two built-in elements.

```
\addPourbaix[name=Fe,element=Fe,c=1e-2,color=blue]
\addPourbaix[name=Cu,element=Cu,c=1e-3,color=red,
  domain label style={pourbaix label,text=red}]
```

2.2 Complete key reference for `\addPourbaix`

Key	Default	Effect and typical value
<code>element</code>	<code>Fe</code>	Element symbol, e.g. <code>Cu</code> . The element must be built in or declared by the user.
<code>c</code>	<code>1e-2</code>	Trace concentration, e.g. <code>1e-4</code> . Its thermodynamic meaning is set by <code>c convention</code> .
<code>c convention</code>	<code>species</code>	Meaning of <code>c</code> : <code>species</code> , <code>element species equal</code> , or <code>element element equal</code> .
<code>color</code>	<code>black</code>	Main drawing color, e.g. <code>blue</code> or <code>red</code> .
<code>species</code>	<code>all</code>	Active species identifiers, e.g. <code>{Fe,Fe2+,Fe(OH)2}</code> .
<code>pHmin, pHmax</code>	<code>0, 14</code>	Computational pH interval, e.g. <code>pHmax=5</code> . Usually match the axis limits.
<code>Emin, Emax</code>	<code>auto</code>	Computational potential interval, e.g. <code>Emin=-1.2,Emax=2</code> .
<code>dpH</code>	<code>0.02</code>	pH scan step before bisection and clipping, e.g. <code>0.005</code> .
<code>labels</code>	<code>true</code>	Master switch for domain labels. Use <code>false</code> to suppress them.
<code>domain labels</code>	<code>formula</code>	Domain label mode: <code>none</code> , <code>formula</code> , <code>letters</code> , or <code>selected</code> .
<code>selected domains</code>	<code>empty</code>	Species shown with formulas in <code>selected</code> mode, e.g. <code>{Fe,Fe^2+,Fe(OH)2}</code> .
<code>domain label phase</code>	<code>neutral</code>	Phase policy for formula labels: <code>neutral</code> , <code>all</code> , or <code>none</code> . Neutral species such as <code>Fe(s)</code> , <code>Fe(OH)2(s)</code> , or <code>H2S(aq)</code> show their phase by default; ions do not. Alias: <code>label phase</code> .
<code>selected domain phase</code>	<code>empty</code>	Optional override for selected formula labels in <code>selected</code> mode. If unset, <code>domain label phase</code> is used. Letter labels never show phases.
<code>label nx, label ny</code>	<code>64, 64</code>	Sampling grid used for label placement, e.g. <code>96</code> .
<code>domain label style</code>	<code>pourbaix label</code>	TikZ style for domain labels; alias: <code>label style</code> .
<code>frontier labels</code>	<code>false</code>	Draw automatic boundary identifiers.
<code>frontier label style</code>	<code>pourbaix frontier label</code>	TikZ node style for boundary labels.
<code>frontier redox labels</code>	<code>true</code>	Enable numeric labels on non-vertical boundaries.
<code>frontier vertical labels</code>	<code>true</code>	Enable lowercase labels on vertical boundaries.
<code>extra x ticks</code>	<code>false</code>	Add exact pH values of vertical boundaries as extra x ticks.

Key	Default	Effect and typical value
<code>extra y ticks</code>	<code>false</code>	Add exact E values of horizontal redox boundaries as extra y ticks.
<code>tick sig figs</code>	<code>4</code>	Significant figures used in extra tick labels.
<code>extra x tick style,</code> <code>extra y tick style</code>	<code>empty</code>	Optional tick-label styles passed to pgfplots for exact boundary ticks, e.g. <code>rotate=90,anchor=east,font=\scriptsize ,text=red</code> .
<code>name</code>	<code>empty</code>	Explicit identifier for later numerical queries; alias for internal <code>id</code> .
<code>line width</code>	<code>0.6pt</code>	Line width used by generated <code>\addplot</code> commands.
<code>forget plot</code>	<code>true</code>	Add <code>forget plot</code> so generated curves do not enter legends.
<code>bbox</code>	<code>true</code>	Clip generated boundaries to the computational box.
<code>water</code>	<code>false</code>	Draw water stability lines.
<code>water preset</code>	<code>aere</code>	Preset for water lines, e.g. <code>standard</code> .
<code>water style</code>	<code>gray dashed</code>	pgfplots style applied to water lines.
<code>water labels</code>	<code>false</code>	Add labels for oxidant/reductant species near the water stability lines.
<code>water label style</code>	<code>font=\scriptsize</code> <code>,...</code>	TikZ node style for water labels.
<code>water label offset</code>	<code>0.10</code>	Vertical offset, in volts. Oxidants are placed above the line and reductants below it.
<code>water label pH low,</code> <code>water label pH high</code>	<code>auto</code>	Optional pH coordinates for the low-pH and high-pH water-label pairs.
<code>water lower oxidant</code> <code>label, water lower</code> <code>reductant label</code>	<code>H⁺, H₂(aq)</code>	Formula contents used near the hydrogen line. These are mhchem contents; do not wrap them in <code>\ce</code> .
<code>water upper oxidant</code> <code>label, water upper</code> <code>reductant label</code>	<code>O₂(aq), H₂O</code>	Formula contents used near the oxygen line; for example, use <code>O₂(g)</code> or <code>HO⁻</code> when desired.

The next section gives copyable examples for the most visible keys. The declaration macros have their own reference section below.

3 Small examples for key groups

3.1 Axis ranges and computational ranges

The pgfplots axis controls what is displayed. The `pHmin`, `pHmax`, `Emin`, and `Emax` keys control what the Lua engine computes. In most documents these should be consistent.

```
\begin{axis}[pourbaix axis,xmin=0,xmax=5,ymin=-1.0,ymax=1.5]
  \addPourbaix[element=V,c=1e-2,pHmin=0,pHmax=5,Emin=-1,Emax=1.5]
\end{axis}
```

3.2 Color, line width, and styles

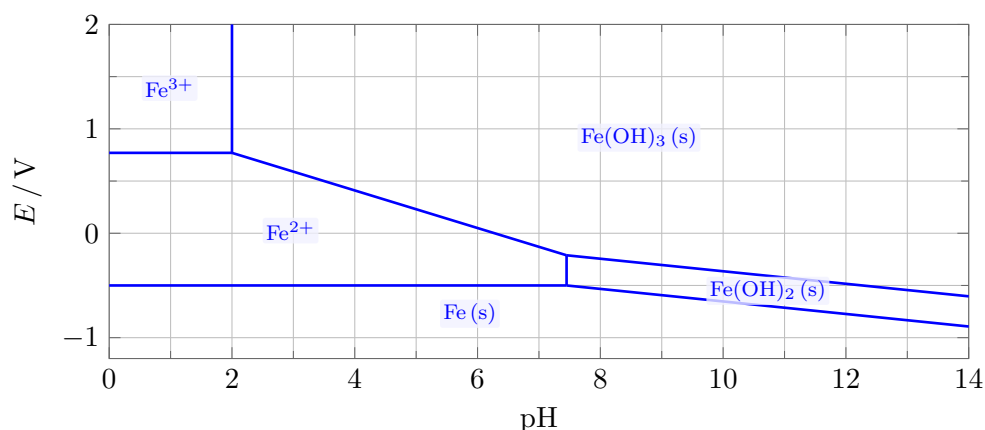


Figure 3: Changing drawing style.

```
\addPourbaix[element=Fe,c=1e-2,color=blue,line width=1pt,  
domain label style={pourbaix label,fill=blue!5,text=blue!60!black}]
```

3.3 Domain label modes

```
% No domain label  
\addPourbaix[element=Fe,c=1e-2,domain labels=none]  
  
% Chemical formulas, using mhchem internally.  
% By default, neutral species show their physical state.  
\addPourbaix[element=Fe,c=1e-2,domain labels=formula]  
  
% Formula labels, with every physical state shown.  
\addPourbaix[element=Fe,c=1e-2,domain labels=formula,  
domain label phase=all]  
  
% Formula labels, with no physical state shown.  
\addPourbaix[element=Fe,c=1e-2,domain labels=formula,  
domain label phase=none]  
  
% Letters A, B, C... sorted left to right and top to bottom  
\addPourbaix[element=Fe,c=1e-2,domain labels=letters]  
  
% Selected formulas; all other domains get letters.  
% Letter labels never show physical states.  
\addPourbaix[element=Fe,c=1e-2,domain labels=selected,  
selected domains={Fe,Fe^2+,Fe(OH)2}]
```

The phase policy applies only to labels that are formulas. The default is `domain label phase=neutral`: neutral species are printed as, for example, Fe(s) , $\text{Fe(OH)}_2\text{(s)}$, or $\text{H}_2\text{S(aq)}$, whereas charged species such as Fe^{2+} or SO_4^{2-} are printed without a phase tag. Use `domain label phase=all` to show phases for all formula labels, and `domain label phase=none` to suppress them. In `domain labels=selected` mode, non-selected domains are letters and therefore never carry physical states.

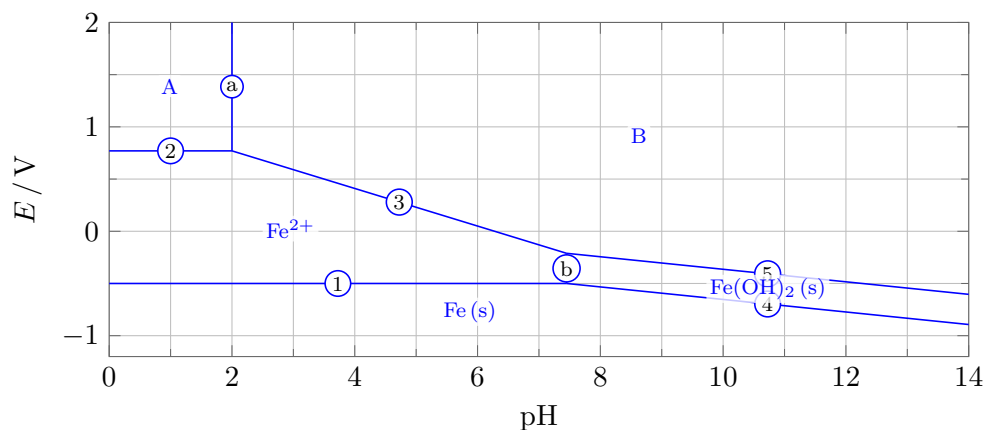


Figure 4: Mixed selected domain labels.

3.4 Frontier labels

The package numbers non-vertical boundaries and uses lowercase letters for vertical ones. The labels are implemented as nodes appended to generated `\addplot` commands.

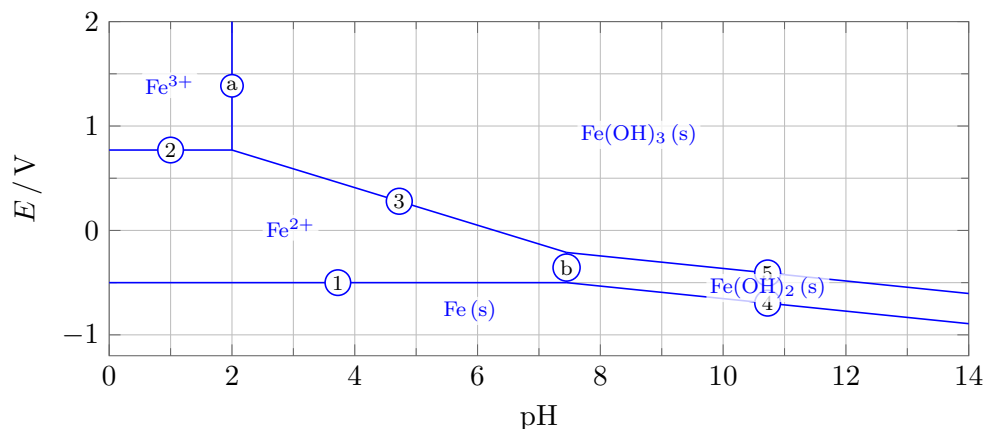


Figure 5: Automatic boundary identifiers.

```
\addPourbaix[element=Fe,c=1e-2,frontier labels=true,
  frontier label style={pourbaix frontier label,fill=white,text=blue}]
```

3.5 Extra ticks

`extra x ticks` prints the exact pH values of vertical boundaries. `extra y ticks` prints the exact E values of horizontal redox boundaries. These are pgfplots extra ticks. If several labels are close to regular ticks or to each other, they may overlap. In that case, use `extra x tick style` and `extra y tick style`. The next example deliberately uses color so that the effect is immediately visible.

```
\addPourbaix[
  element=Fe,c=1e-2,color=blue,
  frontier labels=true,
  extra x ticks=true,
  extra y ticks=true,
  tick sig figs=3,
  extra x tick style={rotate=90,anchor=east,
    font=\scriptsize,text=red},
  extra y tick style={font=\scriptsize,fill=yellow!20,
```

```
inner sep=1pt,text=red}]
```

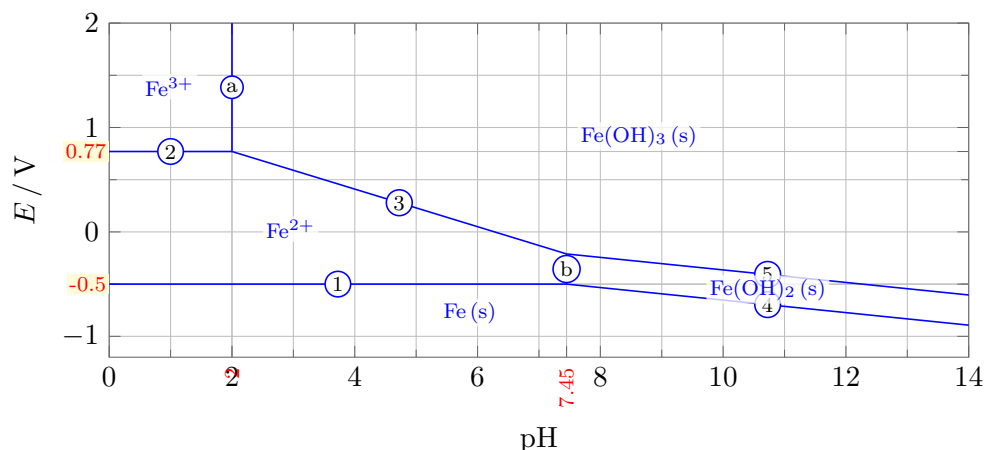


Figure 6: Extra ticks with visible custom styles.

3.6 Water stability lines

```
\addPourbaix[element=Fe,c=1e-2,water=true]
\addPourbaix[element=Fe,c=1e-2,water=true,water preset=standard,
  water style={draw=black!50,densely dashed,line width=.5pt,mark=none}]
```

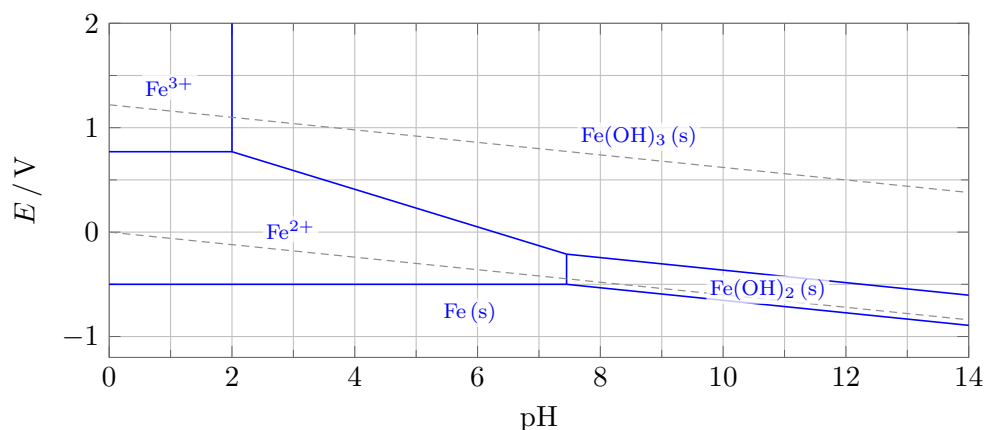


Figure 7: Water lines.

Water labels are treated as an exception to the domain-label system. They are not found by sampling stable domains. Instead, they are placed close to the water boundaries themselves. Oxidant labels are placed above the corresponding line and reductant labels below it. The label keys take raw mhchem contents: write $\text{O}_2(\text{g})$, not $\text{\ce{O2(g)}}$.

```
\addPourbaix[
  element=Fe,c=1e-2,water=true,water labels=true,
  water label style={font=\scriptsize,fill=white,inner sep=1pt},
  water label offset=0.12,
  water lower oxidant label={H+},
  water lower reductant label={H2(g)},
  water upper oxidant label={O2(g)},
  water upper reductant label={H2O},
  water label pH low=1.2,
  water label pH high=12.8]
```

For alkaline conventions, simply change the label contents, for example `water upper reductant label=H0-`.

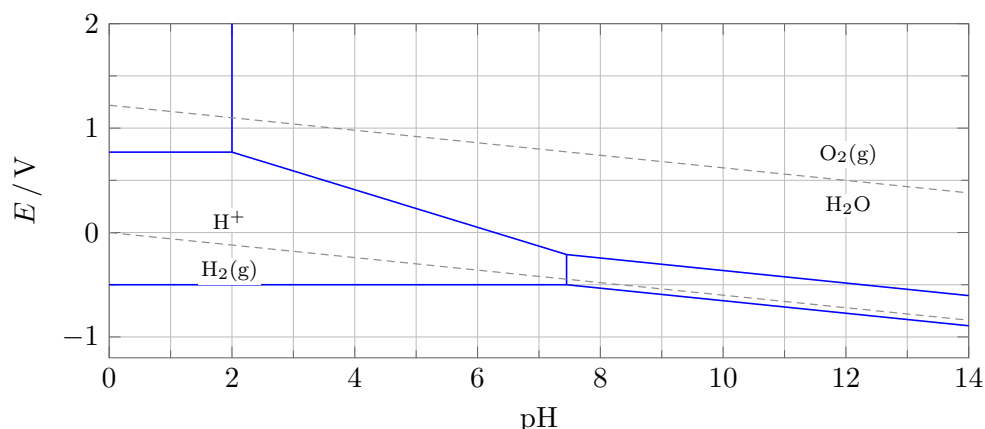


Figure 8: Water lines with customizable oxidant/reductant labels.

3.7 Filtering active species

Use `species` to compute a diagram with only a subset of the species. This is useful for teaching or for checking the role of an individual solid or ion.

```
\addPourbaix[element=Fe,c=1e-2,species={Fe,Fe2+,Fe3+}]
```

3.8 Superposing the same element at different concentrations

Use `name` when the same element appears more than once. The query macros can also use `element` and `c`.

```
\addPourbaix[name=FeHigh,element=Fe,c=1e-2,color=blue]
\addPourbaix[name=FeLow, element=Fe,c=1e-5,color=red]

Slope of boundary 2 at c=1e-2: \pourbaixGetSlope[FeHigh]{2}
Slope of boundary 2 at c=1e-5: \pourbaixGetSlope[FeLow]{2}

% Alternative query syntax:
\pourbaixGetSlope[element=Fe,c=1e-2]{2}
\pourbaixGetSlope[element=Fe,c=1e-5]{2}
```

4 Boundary concentration conventions

The key `c` convention determines the meaning of `c`. This is important for polynuclear species such as $\text{Cr}_2\text{O}_7^{2-}$, $\text{V}_4\text{O}_{12}^{4-}$, or Cl_2 .

4.1 species

This is the historical and default convention. Every aqueous species is assigned activity $a_i = c$. Pure solids, liquids, and gases have unit activity.

```
\addPourbaix[element=Cr,c=1e-2,c convention=species]
```

4.2 element species equal

Here c is interpreted as a total analytical concentration in the followed element. Along an aqueous/aqueous boundary between species A and B , equal species concentrations are imposed:

$$[A] = [B] = \frac{c}{\nu_A + \nu_B},$$

where ν_A and ν_B are the numbers of atoms of the followed element in the two species.

```
\addPourbaix[element=Cr,c=1e-2,c convention={element species equal}]
```

4.3 element element equal

Here c is again a total analytical concentration, but equal element amounts are imposed on the two sides:

$$\nu_A[A] = \nu_B[B] = \frac{c}{2}.$$

Thus $[A] = c/(2\nu_A)$ and $[B] = c/(2\nu_B)$.

```
\addPourbaix[element=Cr,c=1e-2,c convention={element element equal}]
```

For a solid/solution boundary, both analytical conventions reduce to assigning the aqueous species concentration c/ν , where ν is the number of followed-element atoms in that aqueous species.

5 Public macros for database extension

5.1 \PourbaixDeclareElement

Creates or updates an element record.

```
\PourbaixDeclareElement[symbol=X,name={Example metal},metal=X,Emin=-1.2,Emax=1.8]
```

Keys: symbol, element, id, name, metal, Emin, Emax.

5.2 \PourbaixAddSpecies

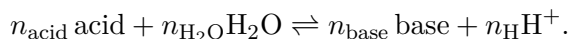
Adds one species to an element.

```
\PourbaixAddSpecies[element=X,id={X2+},formula={X^{2+}},phase=aq,ox state=2,n metal=1]  
\PourbaixAddSpecies[element=X,id={X(OH)2},formula={X(OH)_2},phase=s,ox state=2]
```

Keys: element, id, formula, phase, ox state, color, n metal.

5.3 \PourbaixAddAcidBase

Adds one vertical boundary. The convention is

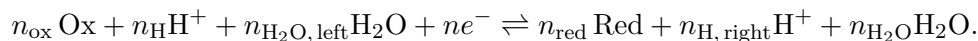


```
\PourbaixAddAcidBase[element=X,acid={X2+},base={X(OH)2},  
nH=2,nH2O=2,pKa=9.7]
```

Keys: element, acid, base, nH, nH2O, pKa, n acid, n base.

5.4 \PourbaixAddRedox

Adds one redox half-reaction. The general convention is



```
\PourbaixAddRedox[element=X,ox={X2+},red=X,E0=-0.38,n=2]  
\PourbaixAddRedox[element=X,ox={X(OH)2},red=X,E0={auto},n=2,nH=2,nH2O=2]
```

Keys: element, ox, red, E0, n, nH, nH right, nH2O, nH2O left, n ox, n red. Use E0=auto for a thermodynamically derived potential.

5.5 \PourbaixAddTemplate

Declaring a new element manually with \PourbaixDeclareElement, \PourbaixAddSpecies, \PourbaixAddAcidBase, and \PourbaixAddRedox is deliberately explicit, but it is also tedious. For a realistic element one must enter species identifiers, formulas, phases, oxidation states, acid/base reactions, redox half-reactions, and several stoichiometric coefficients. \PourbaixAddTemplate performs this tedious bookkeeping for the user.

The template macro parses a list of species, infers the likely declarations, balances acid/base and redox equations, and writes a complete copyable block to the .log. The user then has one remaining task: replace the thermodynamic values marked TODO. The set of required values is determined automatically: vertical constants are requested, low-pH standard potentials between consecutive oxidation states are requested, and thermodynamically redundant potentials are emitted as E0=auto. The expert part explains how this independent set is selected and how the derived constants are reconstructed.

```
\PourbaixAddTemplate[element=V,name={Vanadium}]  
{V02^+, V0^2+, V^3+, V^2+, V205(s), V204(s), V203(s), V4012^4-}
```

The most useful part of the resulting log looks like this:

Listing 1: Representative excerpt from the .log produced by \PourbaixAddTemplate.

```
% =====  
% Skeleton generated by \PourbaixAddTemplate  
% Element : V  
% Thermodynamic mode: only independent constants are requested;  
% chemical equations associated with pKa values are printed;  
% redundant potentials are expressed by thermodynamic cycles.  
% =====  
\PourbaixDeclareElement[symbol=V, name={Vanadium}, metal=V]  
\PourbaixAddSpecies[  
  element=V,  
  id={V02+},  
  formula={V0_2^{+}},  
  phase=aq,  
  ox state=5,  
  n metal=1  
]  
% ... species declarations omitted in this excerpt ...  
  
% Independent vertical constants  
% equation associated with pKa(V3+/V203) :  
% 2 V^3+(aq) + 3 H2O <=> V203(s) + 6 H+  
\PourbaixAddAcidBase[  
  element=V,  
  acid={V3+},  
  base=V203,
```

```

n acid=2,
nH=6,
nH2O=3,
pKa={TODO}
]

% Redox couples: independent and derived potentials
% independent potential: E0(V3+/V2+)
% half-equation associated with E0(V3+/V2+) :
%  $V^{3+}(aq) + e^- \rightleftharpoons V^{2+}(aq)$ 
\PourbaixAddRedox[
    element=V,
    ox={V3+},
    red={V2+},
    E0={TODO},
    n=1
]

% derived potential:  $E0(V2O3/V2+) = E0(V3+/V2+) + 0.03 \cdot pKa(V3+/V2O3)$ 
% half-equation associated with E0(V2O3/V2+) :
%  $V2O3(s) + 6 H^+ + 2 e^- \rightleftharpoons 2 V^{2+}(aq) + 3 H2O$ 
\PourbaixAddRedox[
    element=V,
    ox=V2O3,
    red={V2+},
    E0={auto},
    n=2,
    n red=2,
    nH=6,
    nH2O=3
]

% Independent numerical data to supply:
% pKa(V3+/V2O3) = TODO
% pKa(VO 2+/V2O4) = TODO
% pKa(VO2+/V2O5) = TODO
% pKa(V2O5/V4O12 4-) = TODO
% E0(V3+/V2+) = TODO V
% E0(VO 2+/V3+) = TODO V
% E0(VO2+/VO 2+) = TODO V
%
% Redox potentials derived by thermodynamic cycles:
%  $E0(V2O3/V2+) = E0(V3+/V2+) + 0.03 \cdot pKa(V3+/V2O3)$ 
%  $E0(V2O4/V3+) = E0(VO 2+/V3+) + 0.03 \cdot pKa(VO 2+/V2O4)$ 
%  $E0(V2O5/VO 2+) = E0(VO2+/VO 2+) + 0.03 \cdot pKa(VO2+/V2O5)$ 

```

The main keys are listed below.

Key	Default	Meaning
element, symbol	inferred	followed element symbol
name	element	human-readable name
metal	element	followed element used for atom counting
redox pairs	dominant	emit dominant, adjacent, or all inferred redox pairs
acid base	true	emit inferred acid/base declarations
redox	true	emit inferred redox declarations
comments	true	include explanatory comments and equations in the log
phase default	aq	phase assumed when a species has no explicit phase
check database	true	compare generated values with existing database values when possible

5.6 Loading, validating, and inspecting data

```
\PourbaixLoadData{my-extra-data.tex}
\PourbaixLoadLuaData{my-extra-data.lua}
\PourbaixValidateElement{V}
\PourbaixShowElementData{V}
```

6 Boundary query macros

After a diagram has been drawn, the package stores numerical information about its boundaries. Vertical boundaries are identified by letters; non-vertical boundaries are identified by numbers.

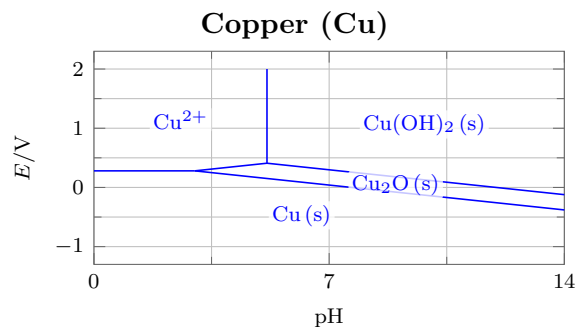
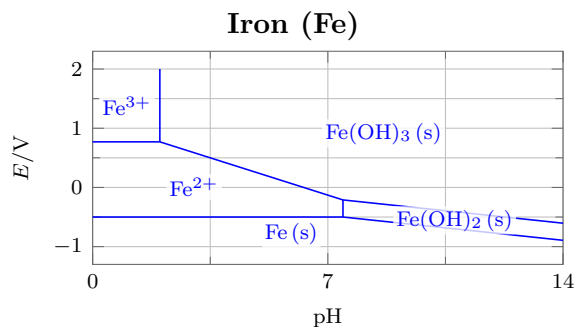
```
\pourbaixGetSlope[element=Fe,c=1e-2]{2}
\pourbaixGetYIntercept[element=Fe,c=1e-2]{2}
\pourbaixGetpH[element=Fe,c=1e-2]{b}
\pourbaixGetEquation[element=Fe,c=1e-2]{2}[4]
```

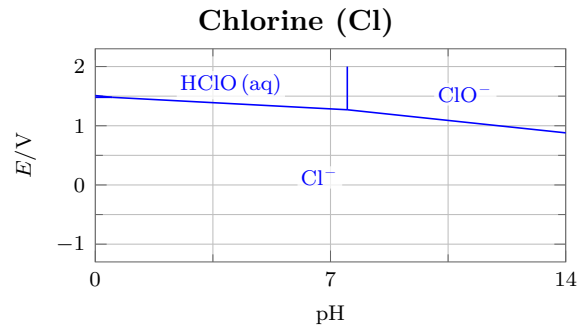
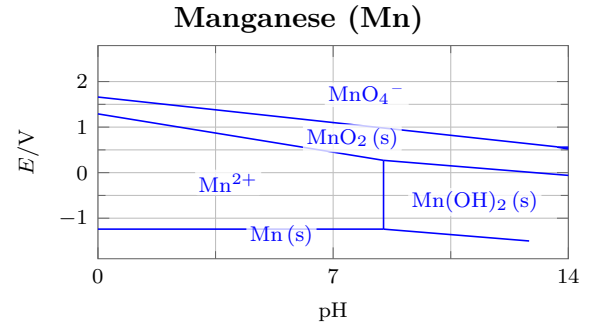
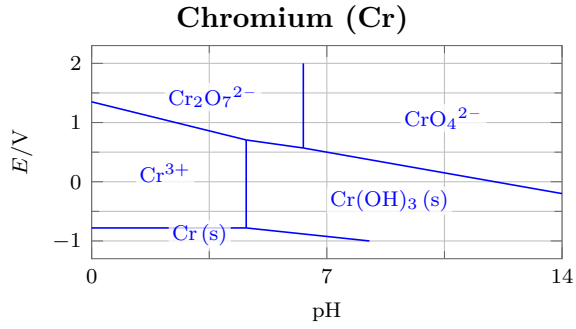
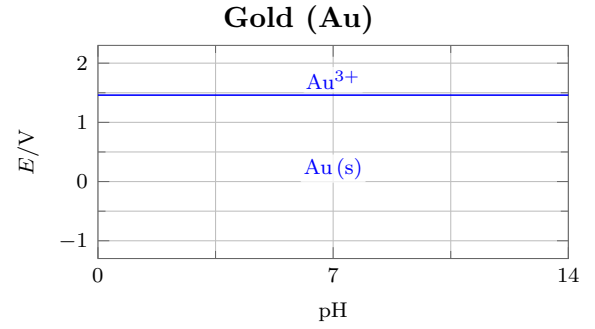
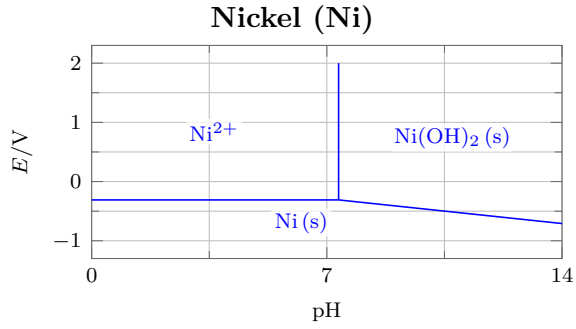
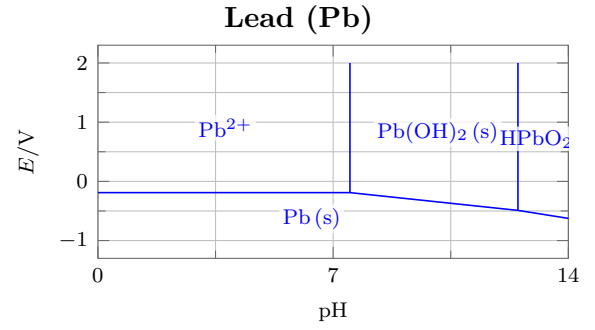
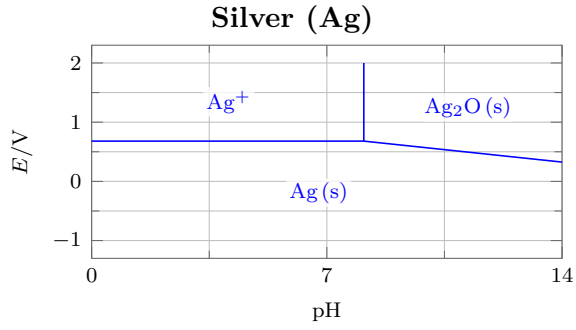
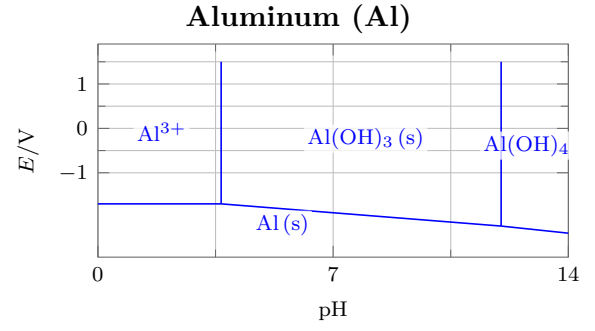
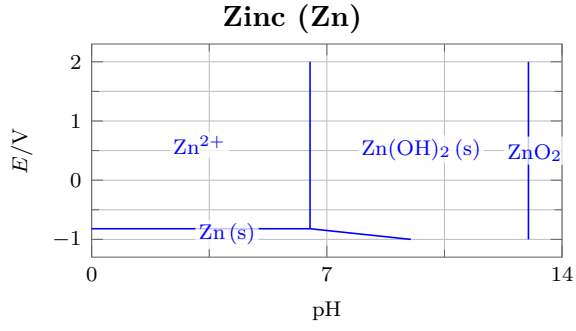
The optional argument can be a simple name, such as **FeHigh**, or a key–value selector such as **element=Fe,c=1e-2,c convention=species**. Use **name** in **\addPourbaix** whenever several diagrams have the same element and concentration.

7 Gallery of diagrams

7.1 Built-in diagrams from pourbaix-data.lua

All diagrams in this gallery use the default domain-label mode, namely formula labels rendered through mhchem.





7.2 Template-based examples

These examples are not built into `pourbaix-data.lua`; they are declared in TeX from data originally generated with `\PourbaixAddTemplate` and then completed with numerical constants. The

diagrams below are generated inside this documentation; they are not included as pre-rendered external PDFs.

7.2.1 Vanadium

The original template request is:

```
\PourbaixAddTemplate[element=V,name={Vanadium}]
  {VO2^+, VO^2+, V^3+, V^2+, V2O5(s), V2O4(s), V2O3(s), V4O12^4-}
```

After copying the generated block from the log and filling the missing constants, the completed data block used by the documentation is:

```
\PourbaixDeclareElement[symbol=V, name={Vanadium}, metal=V]
\PourbaixAddSpecies[
  element=V,
  id={VO2+},
  formula={VO_2^{+}},
  phase=aq,
  ox state=5,
  n metal=1
]
\PourbaixAddSpecies[
  element=V,
  id={VO 2+},
  formula={VO^{2+}},
  phase=aq,
  ox state=4,
  n metal=1
]
\PourbaixAddSpecies[
  element=V,
  id={V2+},
  formula={V^{2+}},
  phase=aq,
  ox state=2,
  n metal=1
]
\PourbaixAddSpecies[
  element=V,
  id={V3+},
  formula={V^{3+}},
  phase=aq,
  ox state=3,
  n metal=1
]
\PourbaixAddSpecies[
  element=V,
  id=V2O5,
  formula={V_2O_5},
  phase=s,
  ox state=5,
  n metal=2
]
\PourbaixAddSpecies[
  element=V,
  id=V2O3,
  formula={V_2O_3},
  phase=s,
  ox state=3,
```

```

    n metal=2
]
\PourbaixAddSpecies[
    element=V,
    id={V4012 4-},
    formula={V_40_{12}^{4-}},
    phase=aq,
    ox state=5,
    n metal=4
]
\PourbaixAddSpecies[
    element=V,
    id=V204,
    formula={V_20_4},
    phase=s,
    ox state=4,
    n metal=2
]
% Independent vertical constants
\PourbaixAddAcidBase[
    element=V,
    acid={V3+},
    base=V203,
    n acid=2,
    nH=6,
    nH2O=3,
    pKa={13.28}
]
\PourbaixAddAcidBase[
    element=V,
    acid={V0 2+},
    base=V204,
    n acid=2,
    nH=4,
    nH2O=2,
    pKa={10.4}
]
\PourbaixAddAcidBase[
    element=V,
    acid={V02+},
    base=V205,
    n acid=2,
    nH=2,
    nH2O=1,
    pKa={-1.4}
]
\PourbaixAddAcidBase[
    element=V,
    acid=V205,
    base={V4012 4-},
    n acid=2,
    nH=4,
    nH2O=2,
    pKa={13}
]
% Redox couples: independent and derived potentials
% independent potential: E0(V3+/V2+)
\PourbaixAddRedox[
    element=V,
    ox={V3+},

```

```

    red={V2+},
    E0={-.35},
    n=1
]
% derived potential:  $E0(V203/V2+) = E0(V3+/V2+) + 0.03 \cdot pKa(V3+/V203)$ 
\PourbaixAddRedox[
    element=V,
    ox=V203,
    red={V2+},
    E0={auto},
    n=2,
    n red=2,
    nH=6,
    nH2O=3
]
% derived potential:  $E0(V0\ 2+/V2+) = 0.5 \cdot E0(V3+/V2+) + 0.5 \cdot E0(V0\ 2+/V3+)$ 
\PourbaixAddRedox[
    element=V,
    ox={V0\ 2+},
    red={V2+},
    E0={auto},
    n=2,
    nH=2,
    nH2O=1
]
% independent potential:  $E0(V0\ 2+/V3+)$ 
\PourbaixAddRedox[
    element=V,
    ox={V0\ 2+},
    red={V3+},
    E0={0.44},
    n=1,
    nH=2,
    nH2O=1
]
% derived potential:  $E0(V204/V2+) = 0.5 \cdot E0(V3+/V2+) + 0.5 \cdot E0(V0\ 2+/V3+) + 0.015 \cdot pKa(V0\ 2+/V204)$ 
\PourbaixAddRedox[
    element=V,
    ox=V204,
    red={V2+},
    E0={auto},
    n=4,
    n red=2,
    nH=8,
    nH2O=4
]
% derived potential:  $E0(V204/V3+) = E0(V0\ 2+/V3+) + 0.03 \cdot pKa(V0\ 2+/V204)$ 
\PourbaixAddRedox[
    element=V,
    ox=V204,
    red={V3+},
    E0={auto},
    n=2,
    n red=2,
    nH=8,
    nH2O=4
]
% derived potential:  $E0(V204/V203) = E0(V0\ 2+/V3+) - 0.03 \cdot pKa(V3+/V203) + 0.03 \cdot pKa(V0\ 2+/V204)$ 

```

```

\PourbaixAddRedox[
  element=V,
  ox=V2O4,
  red=V2O3,
  E0={auto},
  n=2,
  nH=2,
  nH2O=1
]
% derived potential:  $E0(VO2+/V2+) = 0.3333333333*E0(V3+/V2+) + 0.3333333333*E0(VO\ 2+/V3+) + 0.3333333333*E0(VO2+/VO\ 2+)$ 
\PourbaixAddRedox[
  element=V,
  ox={VO2+},
  red={V2+},
  E0={auto},
  n=3,
  nH=4,
  nH2O=2
]
% derived potential:  $E0(VO2+/V3+) = 0.5*E0(VO\ 2+/V3+) + 0.5*E0(VO2+/VO\ 2+)$ 
\PourbaixAddRedox[
  element=V,
  ox={VO2+},
  red={V3+},
  E0={auto},
  n=2,
  nH=4,
  nH2O=2
]
% independent potential:  $E0(VO2+/VO\ 2+)$ 
\PourbaixAddRedox[
  element=V,
  ox={VO2+},
  red={VO\ 2+},
  E0={1},
  n=1,
  nH=2,
  nH2O=1
]
% derived potential:  $E0(V2O5/V2+) = 0.3333333333*E0(V3+/V2+) + 0.3333333333*E0(VO\ 2+/V3+) + 0.3333333333*E0(VO2+/VO\ 2+) + 0.01*pKa(VO2+/V2O5)$ 
\PourbaixAddRedox[
  element=V,
  ox=V2O5,
  red={V2+},
  E0={auto},
  n=6,
  n red=2,
  nH=10,
  nH2O=5
]
% derived potential:  $E0(V2O5/V3+) = 0.5*E0(VO\ 2+/V3+) + 0.5*E0(VO2+/VO\ 2+) + 0.015*pKa(VO2+/V2O5)$ 
\PourbaixAddRedox[
  element=V,
  ox=V2O5,
  red={V3+},
  E0={auto},
  n=4,

```

```

n red=2,
nH=10,
nH2O=5
]
% derived potential:  $E0(V205/V203) = 0.5 \cdot E0(V0\ 2+/V3+) + 0.5 \cdot E0(V02+/V0\ 2+) - 0.015 \cdot pKa(V3+/V203) + 0.015 \cdot pKa(V02+/V205)$ 
\PourbaixAddRedox[
  element=V,
  ox=V205,
  red=V203,
  E0={auto},
  n=4,
  nH=4,
  nH2O=2
]
% derived potential:  $E0(V205/V0\ 2+) = E0(V02+/V0\ 2+) + 0.03 \cdot pKa(V02+/V205)$ 
\PourbaixAddRedox[
  element=V,
  ox=V205,
  red={V0\ 2+},
  E0={auto},
  n=2,
  n red=2,
  nH=6,
  nH2O=3
]
% derived potential:  $E0(V205/V204) = E0(V02+/V0\ 2+) - 0.03 \cdot pKa(V0\ 2+/V204) + 0.03 \cdot pKa(V02+/V205)$ 
\PourbaixAddRedox[
  element=V,
  ox=V205,
  red=V204,
  E0={auto},
  n=2,
  nH=2,
  nH2O=1
]
% derived potential:  $E0(V4012\ 4-/V2+) = 0.3333333333 \cdot E0(V3+/V2+) + 0.3333333333 \cdot E0(V0\ 2+/V3+) + 0.3333333333 \cdot E0(V02+/V0\ 2+) + 0.005 \cdot pKa(V205/V4012\ 4-) + 0.01 \cdot pKa(V02+/V205)$ 
\PourbaixAddRedox[
  element=V,
  ox={V4012\ 4-},
  red={V2+},
  E0={auto},
  n=12,
  n red=4,
  nH=24,
  nH2O=12
]
% derived potential:  $E0(V4012\ 4-/V3+) = 0.5 \cdot E0(V0\ 2+/V3+) + 0.5 \cdot E0(V02+/V0\ 2+) + 0.0075 \cdot pKa(V205/V4012\ 4-) + 0.015 \cdot pKa(V02+/V205)$ 
\PourbaixAddRedox[
  element=V,
  ox={V4012\ 4-},
  red={V3+},
  E0={auto},
  n=8,
  n red=4,
  nH=24,
  nH2O=12
]

```

```

]
% derived potential:  $E_0(V_{4012\ 4-}/V_{203}) = 0.5 \cdot E_0(V_{0\ 2+}/V_{3+}) + 0.5 \cdot E_0(V_{02+}/V_{0\ 2+}) + 0.0075 \cdot$ 
 $pK_a(V_{205}/V_{4012\ 4-}) - 0.015 \cdot pK_a(V_{3+}/V_{203}) + 0.015 \cdot pK_a(V_{02+}/V_{205})$ 
\PourbaixAddRedox[
  element=V,
  ox={V4012 4-},
  red=V203,
  E0={auto},
  n=8,
  n red=2,
  nH=12,
  nH2O=6
]
% derived potential:  $E_0(V_{4012\ 4-}/V_{0\ 2+}) = E_0(V_{02+}/V_{0\ 2+}) + 0.015 \cdot pK_a(V_{205}/V_{4012\ 4-}) +$ 
 $0.03 \cdot pK_a(V_{02+}/V_{205})$ 
\PourbaixAddRedox[
  element=V,
  ox={V4012 4-},
  red={V0 2+},
  E0={auto},
  n=4,
  n red=4,
  nH=16,
  nH2O=8
]
% derived potential:  $E_0(V_{4012\ 4-}/V_{204}) = E_0(V_{02+}/V_{0\ 2+}) + 0.015 \cdot pK_a(V_{205}/V_{4012\ 4-}) -$ 
 $0.03 \cdot pK_a(V_{0\ 2+}/V_{204}) + 0.03 \cdot pK_a(V_{02+}/V_{205})$ 
\PourbaixAddRedox[
  element=V,
  ox={V4012 4-},
  red=V204,
  E0={auto},
  n=4,
  n red=2,
  nH=8,
  nH2O=4
]

```

The diagram is then produced with the following plotting code:

```

\begin{tikzpicture}
\begin{axis}[pourbaix axis,xmin=0,xmax=5,ymin=-1.2,ymax=2]
  \addPourbaix[element=V,c=1e-2,color=blue,domain labels=formula,frontier labels=true,
    water=true];
\end{axis}
\end{tikzpicture}

```

Listing 2: Completed Vanadium data block used by this manual.

```

\PourbaixDeclareElement[symbol=V, name={Vanadium}, metal=V]
\PourbaixAddSpecies[
  element=V,
  id={V02+},
  formula={V0_2^{+}},
  phase=aq,
  ox state=5,
  n metal=1
]
\PourbaixAddSpecies[
  element=V,
  id={V0 2+},

```

```

    formula={VO2+},
    phase=aq,
    ox state=4,
    n metal=1
]
\PourbaixAddSpecies[
    element=V,
    id={V2+},
    formula={V2+},
    phase=aq,
    ox state=2,
    n metal=1
]
\PourbaixAddSpecies[
    element=V,
    id={V3+},
    formula={V3+},
    phase=aq,
    ox state=3,
    n metal=1
]
\PourbaixAddSpecies[
    element=V,
    id=V2O5,
    formula={V2O5},
    phase=s,
    ox state=5,
    n metal=2
]
\PourbaixAddSpecies[
    element=V,
    id=V2O3,
    formula={V2O3},
    phase=s,
    ox state=3,
    n metal=2
]
\PourbaixAddSpecies[
    element=V,
    id={V4O12 4-},
    formula={V4O124-},
    phase=aq,
    ox state=5,
    n metal=4
]
\PourbaixAddSpecies[
    element=V,
    id=V2O4,
    formula={V2O4},
    phase=s,
    ox state=4,
    n metal=2
]
% Independent vertical constants
\PourbaixAddAcidBase[
    element=V,
    acid={V3+},
    base=V2O3,
    n acid=2,
    nH=6,

```

```

    nH2O=3,
    pKa={13.28}
]
\PourbaixAddAcidBase[
    element=V,
    acid={V0 2+},
    base=V204,
    n acid=2,
    nH=4,
    nH2O=2,
    pKa={10.4}
]
\PourbaixAddAcidBase[
    element=V,
    acid={V02+},
    base=V205,
    n acid=2,
    nH=2,
    nH2O=1,
    pKa={-1.4}
]
\PourbaixAddAcidBase[
    element=V,
    acid=V205,
    base={V4012 4-},
    n acid=2,
    nH=4,
    nH2O=2,
    pKa={13}
]
% Redox couples: independent and derived potentials
% independent potential: E0(V3+/V2+)
\PourbaixAddRedox[
    element=V,
    ox={V3+},
    red={V2+},
    E0={-.35},
    n=1
]
% derived potential: E0(V203/V2+) = E0(V3+/V2+) + 0.03*pKa(V3+/V203)
\PourbaixAddRedox[
    element=V,
    ox=V203,
    red={V2+},
    E0={auto},
    n=2,
    n red=2,
    nH=6,
    nH2O=3
]
% derived potential: E0(V0 2+/V2+) = 0.5*E0(V3+/V2+) + 0.5*E0(V0 2+/V3+)
\PourbaixAddRedox[
    element=V,
    ox={V0 2+},
    red={V2+},
    E0={auto},
    n=2,
    nH=2,
    nH2O=1
]

```

```

% independent potential: E0(V0 2+/V3+)
\PourbaixAddRedox[
    element=V,
    ox={V0 2+},
    red={V3+},
    E0={0.44},
    n=1,
    nH=2,
    nH2O=1
]
% derived potential: E0(V2O4/V2+) = 0.5*E0(V3+/V2+) + 0.5*E0(V0 2+/V3+) + 0.015*pKa(V0
    2+/V2O4)
\PourbaixAddRedox[
    element=V,
    ox=V2O4,
    red={V2+},
    E0={auto},
    n=4,
    n red=2,
    nH=8,
    nH2O=4
]
% derived potential: E0(V2O4/V3+) = E0(V0 2+/V3+) + 0.03*pKa(V0 2+/V2O4)
\PourbaixAddRedox[
    element=V,
    ox=V2O4,
    red={V3+},
    E0={auto},
    n=2,
    n red=2,
    nH=8,
    nH2O=4
]
% derived potential: E0(V2O4/V2O3) = E0(V0 2+/V3+) - 0.03*pKa(V3+/V2O3) + 0.03*pKa(V0 2+/
    V2O4)
\PourbaixAddRedox[
    element=V,
    ox=V2O4,
    red=V2O3,
    E0={auto},
    n=2,
    nH=2,
    nH2O=1
]
% derived potential: E0(V02+/V2+) = 0.3333333333*E0(V3+/V2+) + 0.3333333333*E0(V0 2+/V3+)
    + 0.3333333333*E0(V02+/V0 2+)
\PourbaixAddRedox[
    element=V,
    ox={V02+},
    red={V2+},
    E0={auto},
    n=3,
    nH=4,
    nH2O=2
]
% derived potential: E0(V02+/V3+) = 0.5*E0(V0 2+/V3+) + 0.5*E0(V02+/V0 2+)
\PourbaixAddRedox[
    element=V,
    ox={V02+},
    red={V3+},

```

```

E0={auto},
n=2,
nH=4,
nH2O=2
]
% independent potential: E0(V02+/V0 2+)
\PourbaixAddRedox[
  element=V,
  ox={V02+},
  red={V0 2+},
  E0={1},
  n=1,
  nH=2,
  nH2O=1
]
% derived potential: E0(V205/V2+) = 0.3333333333*E0(V3+/V2+) + 0.3333333333*E0(V0 2+/V3+)
+ 0.3333333333*E0(V02+/V0 2+) + 0.01*pKa(V02+/V205)
\PourbaixAddRedox[
  element=V,
  ox=V205,
  red={V2+},
  E0={auto},
  n=6,
  n red=2,
  nH=10,
  nH2O=5
]
% derived potential: E0(V205/V3+) = 0.5*E0(V0 2+/V3+) + 0.5*E0(V02+/V0 2+) + 0.015*pKa(
V02+/V205)
\PourbaixAddRedox[
  element=V,
  ox=V205,
  red={V3+},
  E0={auto},
  n=4,
  n red=2,
  nH=10,
  nH2O=5
]
% derived potential: E0(V205/V203) = 0.5*E0(V0 2+/V3+) + 0.5*E0(V02+/V0 2+) - 0.015*pKa(
V3+/V203) + 0.015*pKa(V02+/V205)
\PourbaixAddRedox[
  element=V,
  ox=V205,
  red=V203,
  E0={auto},
  n=4,
  nH=4,
  nH2O=2
]
% derived potential: E0(V205/V0 2+) = E0(V02+/V0 2+) + 0.03*pKa(V02+/V205)
\PourbaixAddRedox[
  element=V,
  ox=V205,
  red={V0 2+},
  E0={auto},
  n=2,
  n red=2,
  nH=6,
  nH2O=3
]

```

```

]
% derived potential:  $E0(V205/V204) = E0(V02+/V0\ 2+) - 0.03 \cdot pKa(V0\ 2+/V204) + 0.03 \cdot pKa(V02\ +/V205)$ 
\PourbaixAddRedox[
  element=V,
  ox=V205,
  red=V204,
  E0={auto},
  n=2,
  nH=2,
  nH2O=1
]
% derived potential:  $E0(V4012\ 4-/V2+) = 0.3333333333 \cdot E0(V3+/V2+) + 0.3333333333 \cdot E0(V0\ 2+/V3+) + 0.3333333333 \cdot E0(V02+/V0\ 2+) + 0.005 \cdot pKa(V205/V4012\ 4-) + 0.01 \cdot pKa(V02+/V205)$ 
\PourbaixAddRedox[
  element=V,
  ox={V4012\ 4-},
  red={V2+},
  E0={auto},
  n=12,
  n red=4,
  nH=24,
  nH2O=12
]
% derived potential:  $E0(V4012\ 4-/V3+) = 0.5 \cdot E0(V0\ 2+/V3+) + 0.5 \cdot E0(V02+/V0\ 2+) + 0.0075 \cdot pKa(V205/V4012\ 4-) + 0.015 \cdot pKa(V02+/V205)$ 
\PourbaixAddRedox[
  element=V,
  ox={V4012\ 4-},
  red={V3+},
  E0={auto},
  n=8,
  n red=4,
  nH=24,
  nH2O=12
]
% derived potential:  $E0(V4012\ 4-/V203) = 0.5 \cdot E0(V0\ 2+/V3+) + 0.5 \cdot E0(V02+/V0\ 2+) + 0.0075 \cdot pKa(V205/V4012\ 4-) - 0.015 \cdot pKa(V3+/V203) + 0.015 \cdot pKa(V02+/V205)$ 
\PourbaixAddRedox[
  element=V,
  ox={V4012\ 4-},
  red=V203,
  E0={auto},
  n=8,
  n red=2,
  nH=12,
  nH2O=6
]
% derived potential:  $E0(V4012\ 4-/V0\ 2+) = E0(V02+/V0\ 2+) + 0.015 \cdot pKa(V205/V4012\ 4-) + 0.03 \cdot pKa(V02+/V205)$ 
\PourbaixAddRedox[
  element=V,
  ox={V4012\ 4-},
  red={V0\ 2+},
  E0={auto},
  n=4,
  n red=4,
  nH=16,
  nH2O=8
]

```

```
% derived potential: E0(V4012 4-/V204) = E0(V02+/V0 2+) + 0.015*pKa(V205/V4012 4-) -
0.03*pKa(V0 2+/V204) + 0.03*pKa(V02+/V205)
\PourbaixAddRedox[
  element=V,
  ox={V4012 4-},
  red=V204,
  E0={auto},
  n=4,
  n red=2,
  nH=8,
  nH2O=4
]
```

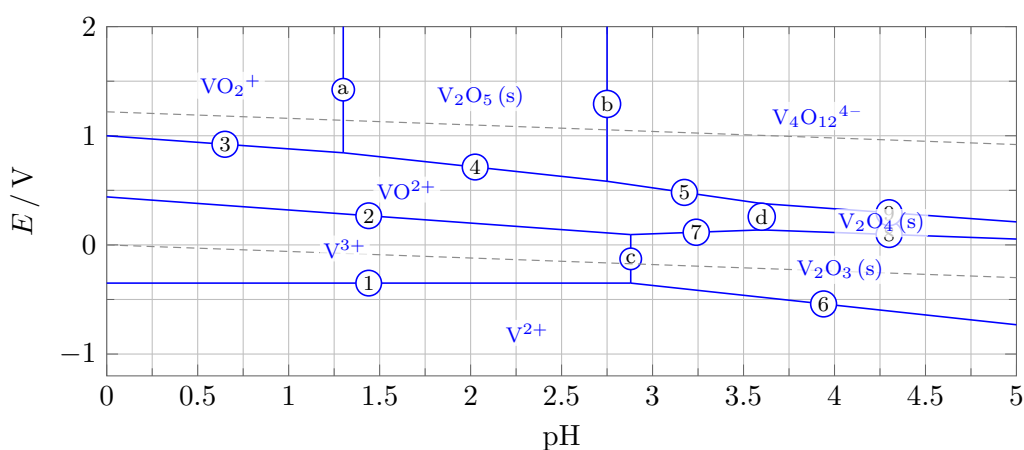


Figure 9: Vanadium diagram generated directly in the manual from template-completed data.

7.2.2 Sulfur

The sulfur data exercise virtual redox boundaries and several acid/base forms of the same oxidation state.

```
\PourbaixAddTemplate[element=S,name={Sulfur}]
{HSO4^-, SO4^2-, S(s), H2S(aq), HS^-, S^2-}
```

After filling the thermodynamic data, the completed data block used by the documentation is:

```
\PourbaixDeclareElement[symbol=S, name={Sulfur}, metal=S]
\PourbaixAddSpecies[
  element=S,
  id=S,
  formula={S},
  phase=s,
  ox state=0,
  n metal=1
]
\PourbaixAddSpecies[
  element=S,
  id=H2S,
  formula={H_2S},
  phase=aq,
  ox state=-2,
  n metal=1
]
\PourbaixAddSpecies[
  element=S,
```

```

    id={HS-},
    formula={HS^{-}},
    phase=aq,
    ox state=-2,
    n metal=1
]
\PourbaixAddSpecies[
    element=S,
    id={S2-},
    formula={S^{2-}},
    phase=aq,
    ox state=-2,
    n metal=1
]
\PourbaixAddSpecies[
    element=S,
    id={HSO4-},
    formula={HSO_4^{-}},
    phase=aq,
    ox state=6,
    n metal=1
]
\PourbaixAddSpecies[
    element=S,
    id={SO4 2-},
    formula={SO_4^{2-}},
    phase=aq,
    ox state=6,
    n metal=1
]
% Independent vertical constants
% equation associated with pKa(HS-/S2-) : HS^-(aq) <=> S^2-(aq) + H+
\PourbaixAddAcidBase[
    element=S,
    acid={HS-},
    base={S2-},
    nH=1,
    nH2O=0,
    pKa={13}
]
% equation associated with pKa(H2S/HS-) : H2S(aq) <=> HS^-(aq) + H+
\PourbaixAddAcidBase[
    element=S,
    acid=H2S,
    base={HS-},
    nH=1,
    nH2O=0,
    pKa={7}
]
% equation associated with pKa(HSO4-/SO4 2-) : HSO4^-(aq) <=> SO4^2-(aq) + H+
\PourbaixAddAcidBase[
    element=S,
    acid={HSO4-},
    base={SO4 2-},
    nH=1,
    nH2O=0,
    pKa={1.9}
]
% Redox couples: independent and derived potentials
% independent potential: E0(S/S2-)

```

```

% half-equation associated with E0(S/S2-) : S(s) + 2 e- <=> S^2-(aq)
\PourbaixAddRedox[
  element=S,
  ox=S,
  red={S2-},
  E0={auto},
  n=2
]
% derived potential: E0(S/HS-) = E0(S/S2-) + 0.03*pKa(HS-/S2-)
% half-equation associated with E0(S/HS-) : S(s) + H+ + 2 e- <=> HS^-(aq)
\PourbaixAddRedox[
  element=S,
  ox=S,
  red={HS-},
  E0={auto},
  n=2,
  nH=1
]
% derived potential: E0(S/H2S) = E0(S/S2-) + 0.03*pKa(H2S/HS-) + 0.03*pKa(HS-/S2-)
% half-equation associated with E0(S/H2S) : S(s) + 2 H+ + 2 e- <=> H2S(aq)
\PourbaixAddRedox[
  element=S,
  ox=S,
  red=H2S,
  E0={0.14},
  n=2,
  nH=2
]
% independent potential: E0(SO4 2-/S)
% half-equation associated with E0(SO4 2-/S) : SO4^2-(aq) + 8 H+ + 6 e- <=> S(s) + 4 H2O
\PourbaixAddRedox[
  element=S,
  ox={SO4 2-},
  red=S,
  E0={auto},
  n=6,
  nH=8,
  nH2O=4
]
% derived potential: E0(HSO4-/S) = E0(SO4 2-/S) - 0.01*pKa(HSO4-/SO4 2-)
% half-equation associated with E0(HSO4-/S) : HSO4^-(aq) + 7 H+ + 6 e- <=> S(s) + 4 H2O
\PourbaixAddRedox[
  element=S,
  ox={HSO4-},
  red=S,
  E0={0.34},
  n=6,
  nH=7,
  nH2O=4
]

```

The diagram is then produced with the following plotting code:

```

\begin{tikzpicture}
\begin{axis}[pourbaix axis,ymin=-1.2,ymax=2]
  \addPourbaix[element=S,c=1e-1,color=blue,domain labels=formula,frontier labels=true,
    water=true];
\end{axis}
\end{tikzpicture}

```

Listing 3: Completed Sulfur data block used by this manual.

```
\PourbaixDeclareElement[symbol=S, name={Sulfur}, metal=S]
\PourbaixAddSpecies[
  element=S,
  id=S,
  formula={S},
  phase=s,
  ox state=0,
  n metal=1
]
\PourbaixAddSpecies[
  element=S,
  id=H2S,
  formula={H_2S},
  phase=aq,
  ox state=-2,
  n metal=1
]
\PourbaixAddSpecies[
  element=S,
  id={HS-},
  formula={HS^{-}},
  phase=aq,
  ox state=-2,
  n metal=1
]
\PourbaixAddSpecies[
  element=S,
  id={S2-},
  formula={S^{2-}},
  phase=aq,
  ox state=-2,
  n metal=1
]
\PourbaixAddSpecies[
  element=S,
  id={HSO4-},
  formula={HSO_4^{-}},
  phase=aq,
  ox state=6,
  n metal=1
]
\PourbaixAddSpecies[
  element=S,
  id={SO4 2-},
  formula={SO_4^{2-}},
  phase=aq,
  ox state=6,
  n metal=1
]
% Independent vertical constants
% equation associated with pKa(HS-/S2-) : HS^-(aq) <=> S^2-(aq) + H+
\PourbaixAddAcidBase[
  element=S,
  acid={HS-},
  base={S2-},
  nH=1,
  nH2O=0,
  pKa={13}
```

```

]
% equation associated with pKa(H2S/HS-) : H2S(aq) <=> HS^-(aq) + H+
\PourbaixAddAcidBase[
    element=S,
    acid=H2S,
    base={HS-},
    nH=1,
    nH2O=0,
    pKa={7}
]
% equation associated with pKa(HSO4-/SO4 2-) : HSO4^-(aq) <=> SO4^2-(aq) + H+
\PourbaixAddAcidBase[
    element=S,
    acid={HSO4-},
    base={SO4 2-},
    nH=1,
    nH2O=0,
    pKa={1.9}
]
% Redox couples: independent and derived potentials
% independent potential: E0(S/S2-)
% half-equation associated with E0(S/S2-) : S(s) + 2 e- <=> S^2-(aq)
\PourbaixAddRedox[
    element=S,
    ox=S,
    red={S2-},
    E0={auto},
    n=2
]
% derived potential: E0(S/HS-) = E0(S/S2-) + 0.03*pKa(HS-/S2-)
% half-equation associated with E0(S/HS-) : S(s) + H+ + 2 e- <=> HS^-(aq)
\PourbaixAddRedox[
    element=S,
    ox=S,
    red={HS-},
    E0={auto},
    n=2,
    nH=1
]
% derived potential: E0(S/H2S) = E0(S/S2-) + 0.03*pKa(H2S/HS-) + 0.03*pKa(HS-/S2-)
% half-equation associated with E0(S/H2S) : S(s) + 2 H+ + 2 e- <=> H2S(aq)
\PourbaixAddRedox[
    element=S,
    ox=S,
    red=H2S,
    E0={0.14},
    n=2,
    nH=2
]
% independent potential: E0(SO4 2-/S)
% half-equation associated with E0(SO4 2-/S) : SO4^2-(aq) + 8 H+ + 6 e- <=> S(s) + 4 H2O
\PourbaixAddRedox[
    element=S,
    ox={SO4 2-},
    red=S,
    E0={auto},
    n=6,
    nH=8,
    nH2O=4
]

```

```
% derived potential: E0(HSO4-/S) = E0(SO4 2-/S) - 0.01*pKa(HSO4-/SO4 2-)
% half-equation associated with E0(HSO4-/S) : HSO4^-(aq) + 7 H+ + 6 e- <=> S(s) + 4 H2O
\PourbaixAddRedox[
  element=S,
  ox={HSO4-},
  red=S,
  E0={0.34},
  n=6,
  nH=7,
  nH2O=4
]
```

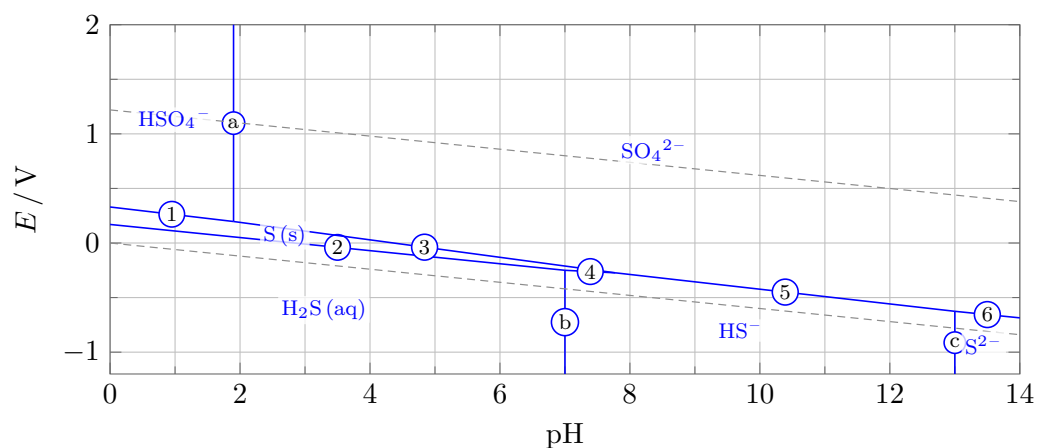


Figure 10: Sulfur diagram generated directly in the manual from template-completed data.

Part II

Expert manual

8 Architecture

pourbaix is organized around a strict separation of concerns. The TeX layer provides a pgfkeys interface, transfers normalized strings to Lua, and receives literal pgfplots/TikZ code. The Lua layer owns the thermodynamic model: activity conventions, acid/base reduction, redox ladder construction, dismutation, derived potentials, virtual boundaries, clipping, and label placement.

This design avoids implementing numerical algorithms in TeX. It also makes the generated pgfplots code intentionally simple: a boundary is a straight segment, represented by two points, optionally followed by a node used as a frontier label.

9 Thermodynamic data model

The main data table is indexed by element symbols. An entry contains

- **name**: human-readable name;
- **metal**: followed element symbol;
- **eRange**: default potential range;
- **species**: map from internal identifiers to species records;
- **acidBaseEquilibria**: vertical-boundary data;
- **redoxCouples**: redox half-reaction data.

A species record contains **formula**, **phase**, **oxState**, optional **color**, and optional **nMetal**. The latter is essential for polynuclear species: $\text{Cr}_2\text{O}_7^{2-}$ has **nMetal**=2; $\text{V}_4\text{O}_{12}^{4-}$ has **nMetal**=4.

10 Activities and concentration conventions

For every redox or acid/base boundary the engine needs effective activities. Pure solids, liquids, and gases have unit activity. Aqueous species are assigned activities through **c convention**. The convention is local to each boundary because a total element concentration must be distributed over the species that define that boundary.

10.1 Species concentration convention

The default convention simply assigns $a_i = c$ to every aqueous species. It is historically convenient and close to many textbook sketches, but it is not an analytical total concentration.

10.2 Analytical concentration with equal species concentrations

For two aqueous boundary species A and B , the engine sets $[A] = [B] = c/(\nu_A + \nu_B)$, where ν_i is the number of followed-element atoms. This convention is useful when the boundary is described as equality of species concentrations.

10.3 Analytical concentration with equal element concentrations

The engine sets $\nu_A[A] = \nu_B[B] = c/2$. This is useful when the boundary convention is equality of analytical element amounts on both sides.

11 Acid/base chains

For a fixed oxidation state, the engine extracts active acid/base equilibria and computes their pH values from

$$\text{pH}_b = \frac{pK + n_{\text{base}} \log a_{\text{base}} - n_{\text{acid}} \log a_{\text{acid}}}{n_{\text{H}}}$$

Boundaries are sorted by increasing pH before reduction. This is important: user input does not have to be ordered. Non-monotone chains are reduced by merging adjacent equilibria. The merged boundary represents disappearance of an intermediate acid/base form at the chosen concentration.

12 Dominant acid/base form

At a given pH and oxidation state, the reduced acid/base chain selects one dominant form. For example, in a chain $\text{H}_2\text{S}/\text{HS}^-/\text{S}^{2-}$, the low-pH form is H_2S , the intermediate form is HS^- , and the high-pH form is S^{2-} . This selection is repeated at all pH samples used by the redox ladder.

13 Redox ladder and dismutation

For a fixed pH, the engine sorts the dominant forms by oxidation state and builds a ladder of stable redox domains. Each new higher oxidation state is accepted only if its transition potential lies above the previous transition. Otherwise the previous state is removed. This is the computational analogue of dismutation or thermodynamic instability of an oxidation state.

The resulting ladder is equivalent to a one-dimensional upper envelope in the potential direction. It determines which species is stable for each interval in E at that pH.

14 Derived and virtual redox couples

Not every visible boundary needs to be explicitly stored. The package distinguishes three cases:

1. Explicit couples supplied with a numerical E0.
2. Derived couples supplied with `E0=auto`. These are reconstructed by solving a thermodynamic linear graph using the actual acid/base constants and redox standard potentials.
3. Virtual couples reconstructed when an intermediate species has been removed from the ladder, but the sum of two half-reactions produces a visible boundary.

The sulfur test is a useful example. After solid sulfur disappears from the redox envelope, visible boundaries involving HS^- and S^{2-} are reconstructed virtually from compatible half-reactions.

15 Boundary extraction

The engine separates chemical existence from geometric visibility. First it samples pH to identify intervals where a given boundary exists chemically. Then the interval endpoints are refined by bisection. Finally, the corresponding straight line is clipped exactly to the rectangle

$$[\text{pH}_{\min}, \text{pH}_{\max}] \times [E_{\min}, E_{\max}].$$

This avoids the earlier failure mode where a redox line stopped at the last sampled visible point instead of at the exact axis boundary.

16 Vertical boundaries

A vertical acid/base boundary is visible only where the species on the left and the species on the right are globally stable at neighboring pH values. The engine evaluates the limiting redox intervals

on both sides of the boundary and intersects them. This prevents vertical boundaries from crossing domains in which neither side is stable.

17 Label placement

Domain labels are placed after the diagram has been computed. The engine samples the plot rectangle on a rectangular grid, groups connected cells with the same stable species, and places one label near each connected component. Formula labels use `mhchem`; letter labels are sorted from left to right and then from top to bottom. Formula labels can show no phase, all phases, or only phases for neutral species. In `domain labels=selected` mode, selected domains use formulas with the chosen phase policy and all other domains use letters; letter labels never carry phase tags.

18 Accessing numerical boundary data

Every drawn diagram is cached under several identifiers: the explicit `name`, the element symbol, and a canonical identifier built from element, concentration, and concentration convention. Query macros therefore work in simple cases and in superpositions of the same element at different concentrations.

For a non-vertical boundary, the cached data are slope and intercept in

$$E = m \text{ pH} + b.$$

For a vertical boundary, the cached data include the constant pH value. `\pourbaixGetEquation` formats these values with a chosen number of significant digits.

19 Template generation and thermodynamic data selection

`\PourbaixAddTemplate` is a data-entry assistant. Its purpose is not to invent thermodynamic constants, but to determine which constants must be supplied and which ones can be deduced from thermodynamic cycles.

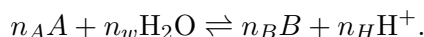
19.1 Parsing and species normalization

The template parser receives a comma-separated list of species. It removes simple `mhchem` or `TeX` wrappers, extracts the terminal phase (`aq`), (`s`), (`l`), or (`g`), extracts a terminal ionic charge, counts atoms, and infers the oxidation state of the followed element. The oxidation-state inference assumes $O = -II$, $H = +I$, and ignores other ligand atoms unless the user later corrects the generated declaration.

The parser then writes normalized species declarations. For instance, `V4012^4-` becomes the internal identifier `V4012 4-`, the display formula `V_{40}{12}^{4-}`, phase `aq`, oxidation state `+V`, and `n metal=4`.

19.2 Vertical constants

For each oxidation state, species are sorted by an acid/base score based mainly on oxygen and hydrogen content. The generator tries to connect neighboring species of the same oxidation state by balanced reactions of the form



Each retained edge becomes one independent vertical constant. The log prints both the declaration and the chemical equation, because the numerical constant supplied by the user must correspond to that exact written reaction.

19.3 Independent redox constants

The generator then chooses low-pH redox potentials between consecutive oxidation states. More precisely, it chooses the root species of each acid/base group and connects roots belonging to adjacent oxidation levels. These potentials are the independent redox values requested as `E0={TODO}`. This matches the common data situation: one usually knows standard potentials such as Fe^{2+}/Fe and $\text{Fe}^{3+}/\text{Fe}^{2+}$, rather than all possible cross-potentials.

19.4 Linear thermodynamic graph

The expert model assigns a formal quantity g_i to each species. It is not an absolute Gibbs energy; it is a convenient linear bookkeeping variable. Each independent acid/base edge contributes an equation

$$n_B g_B - n_A g_A = \frac{RT \ln 10}{F} pK,$$

and each independent redox edge contributes

$$n_{\text{ox}} g_{\text{ox}} - n_{\text{red}} g_{\text{red}} = n E^\circ.$$

One species is chosen as reference with $g = 0$. Propagating these equations gives all reachable species as linear combinations of the independent pK values and independent E° values.

For any other balanced redox half-reaction, the derived standard potential is then

$$E_{\text{derived}}^\circ = \frac{n_{\text{ox}} g_{\text{ox}} - n_{\text{red}} g_{\text{red}}}{n}.$$

The template writes such couples with `E0=auto`. During plotting, `E0=auto` is resolved numerically from the values supplied by the user, including concentration-dependent effective terms when the active concentration convention requires them. If the user possesses a directly tabulated potential for such a derived boundary, they may replace `auto` by a numerical value; the engine will then use the supplied value.

19.5 Selection of emitted couples

The default mode `redox pairs=dominant` emits couples that can contribute to the stable envelope. Non-adjacent or thermodynamically redundant couples are suppressed unless they are needed as automatic or virtual boundaries. `redox pairs=adjacent` is stricter and emits only adjacent oxidation-state pairs. `redox pairs=all` is useful for debugging but may produce many redundant declarations.

19.6 Limitations

The template parser is intentionally modest. It handles ordinary formulas such as $\text{Fe}(\text{OH})_2(\text{s})$, VO_2^+ , $\text{Cr}_2\text{O}_7^{2-}$, and $\text{V}_4\text{O}_{12}^{4-}$. Complex ligands, hydrates with dots, nonstandard charges, and unusual oxidation-state assignments should be checked manually. The log output is a high-quality starting point, not an oracle.

20 Lua function reference

This section documents every function currently defined in `pourbaix.lua`. Public functions are exported as fields of the module table `M`; local functions are implementation details but are documented here because they are important for expert review and future maintenance.

Scope	Name	Arguments	Return	Principle
local	log10	x	Number(s) or table	Computes a base-10 logarithm from the natural logarithm.
local	finite	x	Boolean	Tests whether a numeric value is neither NaN nor infinite.
local	number	x, default	Number(s) or table	Converts an input to a number and falls back to a default.
local	trim	s	Value or side effect	Removes leading and trailing white space.
local	truthy	v, default	Boolean	Normalizes textual booleans such as true, false, yes, no, on, and off.
local	fmt	x	String	Formats a numeric value compactly for generated TeX code.
local	fmt_sig	x, sig	String	Formats a numeric value with a requested number of significant figures.
local	letter_name	n	Value or side effect	Converts an integer into a lowercase boundary label.
local	upper_letter_name	n	Value or side effect	Converts an integer into an uppercase domain label.
local	csv	list	Value or side effect	Joins a list into a comma-separated string.
local	shallow_copy	t	Value or side effect	Copies one level of a Lua table.
local	list_species	elementData	Lua table	Lists species identifiers sorted by oxidation state and identifier.
module	M.active_set	elementData, speciesList	Lua table	Builds the set of species that participate in a diagram.
module	M.normalizeConcentrationConvention	v	Value or side effect	Maps user-facing concentration-convention names onto internal convention identifiers.
local	metal_count	species	Number(s) or table	Returns the number of followed-element atoms carried by a species.
module	M.logActivity	species, cTrace, convention, partnerSpecies	Value or side effect	Computes the logarithm of the effective activity used on a boundary.
module	M.reduceAcidBaseChain	elementData, oxState, cTrace, activeSpeciesSet, convention	Value or side effect	Builds and reduces a monotone acid/base chain for a fixed oxidation state.
module	M.dominantFormAtOxState	elementData, oxState, pH, cTrace, activeSpeciesSet, convention	Value or side effect	Selects the acid/base form stable at a given pH within one oxidation state.
module	M.Eeq	couple, oxSpecies, redSpecies, pH, cTrace, convention	Number(s) or table	Evaluates the Nernst line of a redox couple at a given pH.
module	M.redoxLineCoefficients	couple, oxSpecies, redSpecies, cTrace, convention	Number(s) or table	Computes slope and intercept for a redox boundary.
local	has_intermediate_oxidation_state	elementData, oxId, redId	Boolean	Checks whether an explicitly available oxidation state lies between two species.
module	M.findCouple	elementData, oxId, redId	Value or side effect	Looks up an explicit redox couple by oxidized and reduced species identifiers.
module	M.deriveImplicitEeqCouple	elementData, highId, prevId, ladder, pH, cTrace, convention, activeSpeciesSet	Value or side effect	Reconstructs a virtual redox boundary through a removed intermediate state.
module	M.buildLadder	elementData, pH, cTrace, activeSpeciesSet, convention	Value or side effect	Builds the redox stability ladder at fixed pH by envelope construction.
module	M.dominantSpeciesAt	elementData, pH, E, cTrace, activeSpeciesSet, convention	Value or side effect	Returns the globally stable species at a given pH and E.
module	M.default_range	elementData, range	Number(s) or table	Combines user range options with element defaults.

Scope	Name	Arguments	Return	Principle
local	split_point_groups	points, dpH	Parsed value(s)	Splits sampled redox points into contiguous pH groups.
local	couple_signature	couple	Value or side effect	Creates a stable string describing a redox couple, including virtual-couple data.
local	redox_frontier_key	high, low, couple	Value or side effect	Builds the unique key used to track one redox frontier.
local	has_redox_frontier_at	elementData, pH, cTrace, activeSpeciesSet, convention, key	Boolean	Tests whether a tracked redox frontier exists at a given pH.
local	refine_redox_start	elementData, cTrace, activeSpeciesSet, convention, key, firstPH, pHmin, dpH	Value or side effect	Refines the left endpoint of a redox existence interval by bisection.
local	refine_redox_end	elementData, cTrace, activeSpeciesSet, convention, key, lastPH, pHmax, dpH	Value or side effect	Refines the right endpoint of a redox existence interval by bisection.
local	redox_threshold_at	elementData, couple, pH, cTrace, convention, fallback	Value or side effect	Evaluates a redox threshold from its couple, with a fallback value.
local	ladder_intervals_at	elementData, pHForForms, pHForValues, cTrace, activeSpeciesSet, convention	Value or side effect	Converts a ladder into potential intervals for the stable forms.
local	vertical_frontier_segments	elementData, eq, pHb, cTrace, activeSpeciesSet, range, dpH, convention	Value or side effect	Computes the visible E-intervals of a vertical acid/base boundary.
local	clip_segment_to_rect	x1, y1, x2, y2, xmin, xmax, ymin, ymax	Value or side effect	Clips a straight segment to the rectangular plotting window.
module	M.computeFrontiers	elementData, cTrace, activeSpeciesSet, range, dpH, convention	Boundary table	Computes all visible redox and vertical frontiers for one diagram.
module	M.waterFrontiers	pO2, pH2	Boundary table	Returns the two water-stability lines for a chosen gas-pressure preset.
module	M.mhchem_formula	formula	Value or side effect	Converts an internal formula into mhchem syntax.
module	M.label_text	elementData, id, showPhase	String	Builds the TeX label for a species, optionally with a phase tag.
local	canonical_domain_id_text	s	String	Normalizes a user-written species name for selected-domain labels.
local	split_selected_domain_list	s	Parsed value(s)	Splits the selected-domain list while respecting braces and parentheses.
local	selected_domain_set	elementData, elementSymbol, list	Lua table	Builds the set of domains selected for formula labels.
module	M.label_positions	elementData, cTrace, activeSet, range, nx, ny, minCells, convention	Lua table	Samples the diagram and places domain labels by connected components.
module	M.annotateFrontiers	elementData, cTrace, fr, convention	Boundary table	Assigns visible names and cached metadata to frontiers.
local	canonical_registry_id	element, cTrace, convention	String	Builds the canonical registry id from element, concentration, and convention.
local	parse_registry_selector	sel	Parsed value(s)	Parses the optional selector used by numerical query macros.
module	M.storeFrontierRegistry	id, element, cTrace, fr, byName, convention	Value or side effect	Stores boundary metadata under name, element, and canonical identifiers.
local	registry_entry	id	Value or side effect	Retrieves the cached frontier registry entry matching a selector.
local	registry_value	entry, name, field	Value or side effect	Reads one numeric field from one registered boundary.
module	M.tex_get_value	id, name, field, sig	TeX output or side effect	TeX bridge for slope, intercept, or pH queries.

Scope	Name	Arguments	Return	Principle
module	M.tex_get_equation	id, name, sig	TeX output or side effect	TeX bridge formatting a boundary equation.
local	append_unique	list, labels, value, label, tol	Value or side effect	Appends a tick value unless an existing value is already within tolerance.
module	M.tex_reset_axis_ticks		TeX output or side effect	Clears the accumulated extra tick cache for a new axis.
local	merge_ticks	dst_values, dst_labels, src_values, src_labels	Lua table	Merges tick values and labels into the current axis cache.
local	emit_extra_ticks	axis, values, labels	Side effect	Writes pgfplots extra tick options for one axis direction.
local	emit_current_axis_extra_ticks	new_x_values, new_x_labels, new_y_values, new_y_labels	Side effect	Emits accumulated extra x and extra y tick options.
local	tex_escape_message	s	TeX output or side effect	Escapes text for diagnostic messages sent through TeX.
local	lua_warn	msg	Side effect	Writes a package warning from Lua.
local	lua_info	msg	Side effect	Writes an informational message from Lua.
module	M.available_elements		Value or side effect	Lists built-in or user-declared element symbols.
module	M.available_elements_string		String	Formats the available-element list as a string.
local	ensure_element	symbol	Value or side effect	Creates an element table if needed and returns it.
local	parse_optional_number	v	Parsed value(s)	Parses an optional numeric key; empty values yield nil.
local	parse_required_number	v, default	Parsed value(s)	Parses a required numeric key and falls back to a default.
module	M.declare_element	spec	Boolean or status information	Adds or updates an element metadata record.
module	M.add_species	elementSymbol, spec	Boolean or status information	Adds one species record to an element.
module	M.add_acid_base	elementSymbol, eq	Boolean or status information	Adds one acid/base equilibrium to an element.
module	M.add_redox	elementSymbol, couple	Boolean or status information	Adds one redox couple, including numerical or automatic E0 handling.
local	merge_element	dst, src	Lua table	Merges one element data table into another.
module	M.merge_data	tbl	Lua table	Merges a user data table into the main database.
module	M.load_user_data	filename	Boolean or status information	Loads a Lua file returning user thermodynamic data.
module	M.validate_element	symbol	Boolean or status information	Checks the internal consistency of an element data record.
module	M.tex_validate_element	symbol	TeX output or side effect	TeX bridge for validation diagnostics.
local	dump_value	v, indent, seen	Value or side effect	Serializes a Lua value recursively for log inspection.
module	M.tex_show_element_data	symbol	TeX output or side effect	Writes the normalized element data table to the log.
local	gcd_int	a, b	Value or side effect	Computes an integer greatest common divisor.
local	lcm_int	a, b	Value or side effect	Computes an integer least common multiple.
local	split_top_level_commas	s	Parsed value(s)	Splits a comma list without breaking groups or parentheses.
local	parse_charge	raw	Parsed value(s)	Parses a terminal ionic charge.
local	strip_tex_formula	s	Value or side effect	Removes simple TeX and mhchem markup from a formula string.
local	split_phase_and_charge	raw, default_phase	Parsed value(s)	Extracts phase and charge from a template species.

Scope	Name	Arguments	Return	Principle
local	parse_atom_counts	formula	Parsed value(s)	Counts atoms in a formula, including parenthesized groups.
local	infer_oxidation_state	element, counts, charge	Inferred record or nil	Infers the oxidation state of the followed element from atom counts and charge.
local	latex_formula_from_core	core, charge_label	Value or side effect	Builds a display formula with TeX subscripts and superscripts.
local	id_from_core	element, core, charge_label	Value or side effect	Builds the internal species identifier from formula core and charge.
local	parse_template_species	raw, element, default_phase	Parsed value(s)	Parses one species supplied to the template generator.
local	infer_element_from_species	items	Inferred record or nil	Chooses a likely followed element from a species list.
local	fmt_int_or_num	x	String	Formats an integer-like coefficient without a decimal point.
local	command_quote_id	id	Value or side effect	Adds braces around an identifier when TeX key syntax requires it.
local	template_line	line	Side effect	Writes one line of generated template code to the terminal and log.
local	species_label	sp	String	Returns a compact label for a template species.
local	species_equation_label	sp, with_phase	String	Formats a species for a generated chemical equation.
local	term_for_equation	coef, label	Value or side effect	Formats one stoichiometric term in a generated equation.
local	join_equation_terms	terms	Value or side effect	Joins nonzero equation terms with plus signs.
local	acid_base_equation_text	ab	String	Builds the text of a generated acid/base equation.
local	redox_equation_text	r	String	Builds the text of a generated redox half-equation.
local	ab_label	ab	String	Builds the symbolic label of an acid/base constant.
local	redox_label	r	String	Builds the symbolic label of a redox potential.
local	infer_acid_base	a, b	Inferred record or nil	Balances an acid/base equilibrium between two same-oxidation-state species.
local	infer_redox	ox, red	Inferred record or nil	Balances a redox half-reaction between two species.
local	emit_keyval_command	name, parts	Side effect	Writes a generated pgfkeys-style command block.
local	emit_species_template	element, sp	Side effect	Writes a generated species declaration.
local	emit_acidbase_template	element, ab, pka_text	Side effect	Writes a generated acid/base declaration and its equation comment.
local	emit_redox_template	element, r, e0_text, note	Side effect	Writes a generated redox declaration and its half-equation comment.
local	expr_zero		Linear expression or number	Manipulates linear expressions in independent pK and E0 variables.
local	expr_const	c	Linear expression or number	Manipulates linear expressions in independent pK and E0 variables.
local	expr_var	name	Linear expression or number	Manipulates linear expressions in independent pK and E0 variables.
local	expr_clone	a	Linear expression or number	Manipulates linear expressions in independent pK and E0 variables.
local	expr_add	a,b	Linear expression or number	Manipulates linear expressions in independent pK and E0 variables.

Scope	Name	Arguments	Return	Principle
local	<code>expr_sub</code>	<code>a, b</code>	Linear expression or number	Manipulates linear expressions in independent pK and E0 variables.
global	<code>expr_scale</code>	<code>a, s</code>	Linear expression or number	Manipulates linear expressions in independent pK and E0 variables.
local	<code>expr_eval</code>	<code>a, values</code>	Linear expression or number	Manipulates linear expressions in independent pK and E0 variables.
local	<code>fmt_coef</code>	<code>x</code>	String	Formats a coefficient in a symbolic linear expression.
local	<code>expr_format</code>	<code>a</code>	Linear expression or number	Manipulates linear expressions in independent pK and E0 variables.
local	<code>pair_key</code>	<code>a, b</code>	Value or side effect	Builds a stable key for a pair of template species.
local	<code>acid_base_score</code>	<code>sp</code>	Value or side effect	Scores species by oxygen and hydrogen content to sort acid/base forms.
local	<code>build_acid_base_forest</code>	<code>species</code>	Lua table	Builds the acid/base spanning forest used to select independent vertical constants.
local	<code>add_ab_equation</code>	<code>eqs, ab</code>	Value or side effect	Adds an acid/base linear equation to the template graph.
local	<code>add_redox_equation</code>	<code>eqs, r, label</code>	Value or side effect	Adds a redox linear equation to the template graph.
local	<code>solve_g_expressions</code>	<code>species, ab_edges, independent_redox</code>	Value or side effect	Solves symbolic species free-energy expressions from the independent graph.
local	<code>build_independent_redox</code>	<code>species, forest</code>	Lua table	Selects low-pH redox potentials between consecutive oxidation states.
local	<code>should_emit_redox_pair</code>	<code>mode, forest, high, low, adjacentOx</code>	Value or side effect	Decides whether a template redox pair should be emitted.
local	<code>find_database_redox_E0</code>	<code>elementSymbol, ox, red</code>	Record or nil	Looks up a database E0 used for template self-checks.
local	<code>find_database_pKa</code>	<code>elementSymbol, acid, base</code>	Record or nil	Looks up a database pK value used for template self-checks.
local	<code>build_runtime_species_objects</code>	<code>elementSymbol, elementData</code>	Lua table	Reconstructs template-like species records from runtime element data.
local	<code>solve_effective_species_g</code>	<code>elementSymbol, elementData, cTrace, convention</code>	Value or side effect	Solves numerical species free-energy values with concentration-dependent terms.
local	<code>add_missing_adjacent_auto_couples</code>	<code>elementData, elementSymbol</code>	Value or side effect	Adds automatic adjacent couples needed by the redox ladder.
module	<code>M.resolve_auto_redox_E0s</code>	<code>elementData, elementSymbol, cTrace, convention</code>	Value or side effect	Resolves E0=auto couples before plotting, using current c and convention.
module	<code>M.tex_add_template</code>	<code>opts</code>	TeX output or side effect	TeX bridge for writing the template-generated skeleton to the log.
local	<code>tex_bool</code>	<code>v, default</code>	TeX output or side effect	Normalizes TeX boolean key values.
local	<code>get_range_from_opts</code>	<code>elementData, opts</code>	Value or side effect	Builds the computational range from addPourbaix options.
local	<code>sanitize_color</code>	<code>c</code>	Value or side effect	Sanitizes color strings for generated TeX code.
module	<code>M.tex_add_pourbaix</code>	<code>opts</code>	TeX output or side effect	Main TeX bridge: computes and emits the pgfplots drawing code.

21 Design choices

- Public syntax is pgfkeys-based because it is familiar in TikZ/pgfplots workflows.
- Lua owns the chemistry and numerical work.
- pgfplots is used only as a high-quality rendering backend.
- Boundaries are straight segments; two points are sufficient.
- Labels and ticks are optional annotations; they do not affect thermodynamic construction.
- User-defined data are merged with the built-in database, but the built-in table remains ordinary Lua data.

22 License, acknowledgments, and disclaimer

This work may be distributed and/or modified under the conditions of the LaTeX Project Public License, either version 1.3c of this license or, at your option, any later version. The work has the LPPL maintenance status “maintained”. The Current Maintainer is Christophe Jorssen.

The first exploratory stage of this package was informed by the interactive web animation by Thomas Roy, available at <https://phyzik.store/?anim=pourbaix>. In particular, the contents of `pourbaix-data.lua` originate, in the early development versions, from a raw extraction of Thomas Roy’s thermodynamic dataset and species definitions. The public package architecture, Lua engine, pgfplots interface, and documentation were then reworked into the present `pourbaix` package.

Development disclaimer: this package was “vibe-coded” by Christophe Jorssen (christophe.jorssen@gmail.com) with the assistance of ChatGPT 5.5. The package is intended as a technical and pedagogical tool. Users should verify thermodynamic data, concentration conventions, and generated diagrams for their intended use.